NOTE PERSONNELLE · TECHNIQUE

Comment fonctionne JWT dans SmartTrans-AI

Explication fonctionnelle et technique, calée sur le code réel (AuthService, JwtService, JwtAuthenticationFilter, React).

1. Vue fonctionnelle (ce que voit l'utilisateur)

JWT (JSON Web Token) sert à rester connecté sans session serveur. Après un login réussi, l'API donne un jeton. Le navigateur le range et le renvoie à chaque appel. L'API n'a pas besoin de mémoriser « qui est en ligne » dans une session : elle relit le jeton.

Parcours côté métier

- L'utilisateur saisit e-mail et mot de passe sur /login.
- Le serveur vérifie le compte (hachage BCrypt, compte activé).
- Il émet un jeton Bearer + le profil (id, nom, rôles).
- React stocke le jeton dans localStorage (`smarttrans.accessToken`).
- Chaque requête API (sauf le login) part avec `Authorization: Bearer …`.
- Si le jeton est absent, expiré ou falsifié : 401. Si le rôle ne suffit pas : 403.

Les rôles voyagent dans le jeton (ADMIN, TRANSPORT_MANAGER, DRIVER). L'interface s'en sert pour les menus ; l'API s'en sert pour vraiment bloquer les actions.

2. Qu'est-ce qu'un JWT ?

Un JWT est une chaîne en trois parties séparées par des points : header.payload.signature. Header et payload sont en Base64URL (lisibles, pas chiffrés). La signature HMAC-SHA empêche de modifier le contenu sans connaître le secret.

Partie	Rôle
Header	Algorithme (ici HMAC-SHA à partir du secret) et type JWT.
Payload (claims)	Données : sujet, e-mail, rôles, dates iat / exp.
Signature	Empreinte du header+payload avec le secret serveur. Vérifie l'intégrité.

Claims utilisés dans SmartTrans-AI

Claim	Contenu
sub	UUID de l'utilisateur (principal.getId()).
email	Adresse de connexion (username Spring).
roles	Liste sans préfixe ROLE_ : ADMIN, TRANSPORT_MANAGER, DRIVER.
iat / exp	Émis maintenant ; expire après app.jwt.expiration-ms (défaut 86 400 000 ms = 24 h).

3. Chaîne technique dans le projet

A. Login — émission

POST /api/v1/auth/login est public (pas de Bearer). AuthServiceImpl appelle l'AuthenticationManager (e-mail + mot de passe). Si OK, JwtService.generateToken construit le JWT avec JJWT (signWith(secretKey), clé HMAC dérivée de app.jwt.secret). La réponse JSON contient accessToken, tokenType: Bearer, expiresInMs et le profil. Un événement d'audit LOGIN est enregistré.

B. Appels suivants — filtre

JwtAuthenticationFilter (OncePerRequestFilter) s'exécute avant le filtre username/password. S'il n'y a pas d'en-tête Bearer, ou si l'URL commence par /api/v1/auth/, il laisse passer sans poser d'authentification. Sinon il coupe le préfixe Bearer, appelle jwtService.isValid (parse + vérif signature + expiration). Si valide : extraire l'e-mail, recharger l'utilisateur (CustomUserDetailsService), poser un UsernamePasswordAuthenticationToken dans le SecurityContext. Les autorités Spring sont alors disponibles pour @PreAuthorize.

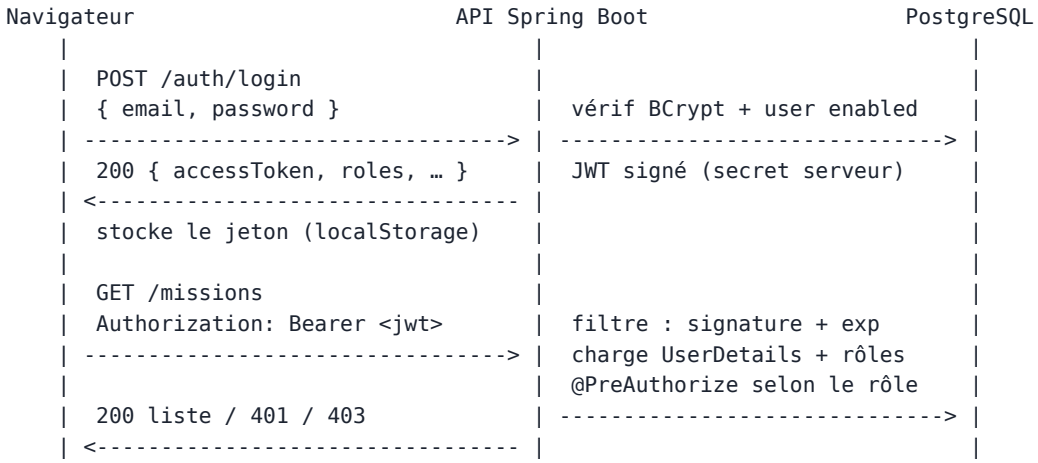
C. Politique de sécurité

- Session STATELESS : pas de cookie de session JSESSIONID pour l'API.
- CSRF désactivé (API jeton, pas de formulaire cookie classique).
- Mot de passe : BCryptPasswordEncoder.
- Routes ouvertes : /api/v1/auth/**, health, swagger, H2.
- Tout le reste : authenticated(), puis rôles au niveau des contrôleurs.
- 401 : pas / plus authentifié (JwtAuthenticationEntryPoint). 403 : authentifié mais rôle insuffisant.

D. Front React

Le client HTTP Axios (api/client.ts) attache le Bearer si un jeton existe. Le login ne doit pas envoyer un ancien jeton (d'où l'exclusion des routes /auth/ dans le filtre). Déconnexion = suppression du localStorage.

4. Schéma du flux



5. Points à retenir (soutenance / debug)

- Le payload est lisible : ne jamais y mettre le mot de passe.
- La confiance vient de la signature + secret, pas du fait que le JSON soit « caché ».
- Le secret doit être assez long (HMAC-SHA, ≥ 256 bits) et hors Git en prod (JWT_SECRET).

- Durée de vie 24 h par défaut : après expiration, il faut se reconnecter (pas de refresh token dans ce livrable).
- Les rôles dans le jeton sont une photo à l’instant du login : un changement de rôle en base n’est pris en compte qu’au prochain login (le filtre recharge toutefois l’utilisateur par e-mail pour les autorités actuelles).

Classes clés : `JwtService`, `JwtAuthenticationFilter`, `AuthServiceImpl`, `SecurityConfig`, `tokenStorage.ts`.